

六軸方向の同時操作と触覚フィードバックを加えた ゲームエンジンエディタ拡張機能の開発

25M5326 勝山 莞太 指導教員 今村 弘樹

2026 年 7 月 8 日

Abstract: Recent video games are developed by a set of tools called game engines. In the level design of these engines, the target objects are three-dimensional objects, but users of current game engines operate them with a two-dimensional mouse cursor. Therefore, even for simple objects, extra steps are required, such as moving the viewpoint for manipulation. To prevent such problems, a previous study replaced the mouse cursor with a pen-type haptic device that allows users to manipulate the level editor in the game engine. Using this device, objects can be moved and rotated in each axis, preventing unexpected floating or passing between objects. This research will build on that prior research to address the remaining issues and extend the functionality.

Keywords: game design, haptic device, extend the editor

1 はじめに

1.1 研究の背景

近年のビデオゲーム開発では、ゲームエンジンというゲーム開発に向けた機能を併せ持ったツールによって開発されることが多い。[1] ゲームエンジンは基本的なコンパイルや、物理演算、レベルデザインなどの機能を持ち、高度な設計が要求されるゲーム開発を支えている。それらの機能の中でレベルデザイン（ステージデザイン）の機能では、3D オブジェクトをマウスカーソルで操作し、オブジェクトを配置する。実際に我々が現実世界でものを配置する際は視点を動かさずとも遠近感を掴むことができるが、モニター上では 3D オブジェクトの遠近感が把握しにくい。そのため、それぞれの軸方向にカメラを回転や移動をする手間がかかる。また、これらの視点移動をしなかった場合オブジェクトの意図しないすり抜けや浮遊にも繋がる。オブジェクトの浮遊とすり抜けを図 1 に示す。

1.2 研究の目的

先行研究では、3Dsystems 社のハプティクスデバイス Touch[2] を用いて、ゲームエンジン上に X,Y,Z 軸とその回転軸を合わせた 6 軸方向の同時操作と触覚フィードバックを加えた拡張機能を開発した。こ

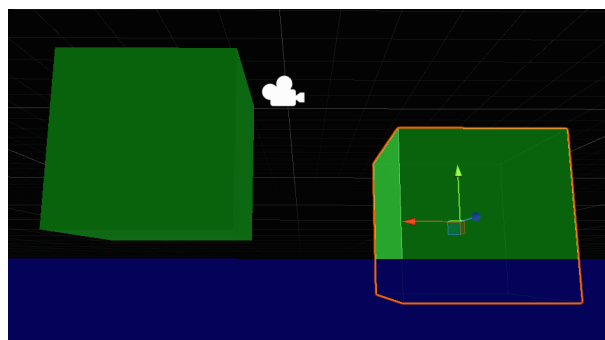


図 1 オブジェクトの浮遊とすり抜け

れは、同時操作によって視点移動の手間を省き、また、物体間の衝突をユーザーに触覚フィードバックすることで意図しないすり抜けや浮遊を防止している。

本研究では、先行研究に残された課題点を解決する。取り組んだ課題点は以下の三つである。

1. 任意の形状のオブジェクトを操作可能とする。
2. 任意の形状のオブジェクト同士の衝突判定処理を可能とする。

これらの課題点を解決することで、ゲーム制作においてより活用しやすくなると考える。

2 提案手法

2.1 使用ツール

2.1.1 Unity

Unity[3] とは, UnityTechnologies が開発しているゲームエンジンである. (図 2) 基本的なコンパイル, 物理演算, レベルデザインの機能を持ち, 現在最も多くの現場で使用されているゲームエンジンである. 公式に機能の拡張を推奨しており, 本研究で行う開発にも対応している.

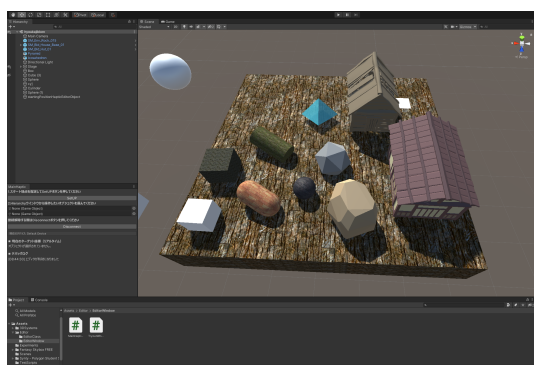


図 2 Unity

2.1.2 Touch

Touch とは, 3DSYSTEMS 社が開発したペン型のハプティクスデバイスである. (図 3) ユーザーはペンの部分を持ち, 動かすことでオブジェクトと連動し操作できる. 本研究で必要な 6 軸方向の同時操作と触覚フィードバックの二つの機能が備わっている.



図 3 Touch

2.2 先行研究の概要

2.2.1 システムの流れ

システム全体のフローチャートを図 4 に示す. オブジェクトの移動前に座標を保持し, 新たな座標に設定する前に扱う. また, 衝突情報は取得後リストにまとめて Touch に送信し, 触覚フィードバックを実現する.

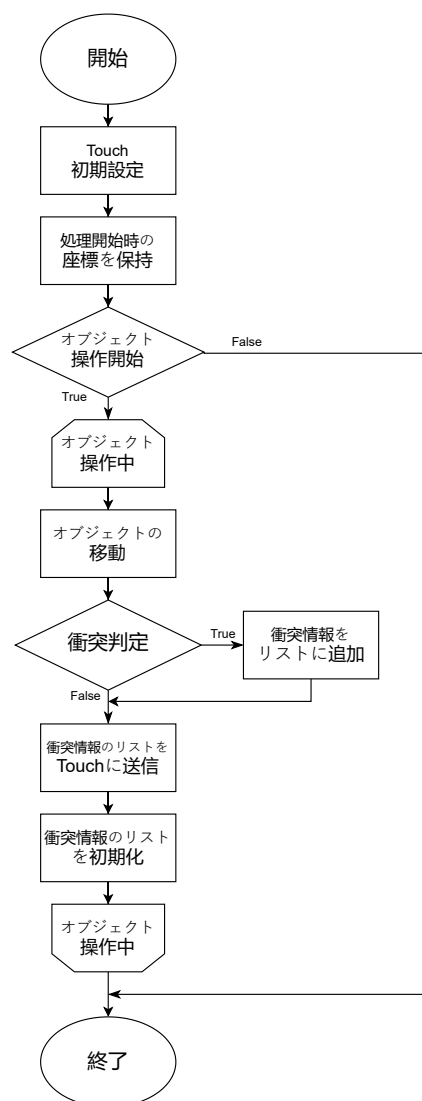


図 4 全体のフローチャート

2.2.2 3D オブジェクトの移動と衝突

Touch 上で操作した際の座標の変動数は Unity 上の座標で図った時より大きいので, Touch 上の座標を Unity に適用させる際は定数 0.1 との積を取る. また, Touch を操作せずペンを格納している状態の座標は $(0, -65.5, -88.1)$ である. この座標を原点として扱うため, 定ベクトル a とし Touch から取得する座標と常に差を取る. 先行研究で実装されていたのは球形の操作と直方体との衝突判定処理である. 球体の衝突判定では Unity の関数である *Physics.OverlapSphere* 関数を使用していた. 二つの引数から球形の範囲を設定し, そこに触れたすべてのコライダーを取得する関数である. 直方体オブジェクトとの衝突判定処理について, Touch に触覚フィードバックを付与するには, 「デバイスから

力を加える Touch 上の座標」と「デバイスが加える力の向き」が必要になる。この 2 つの情報を得るため、直方体との衝突では Plane 関数を使用していた。Plane クラスは任意の 3 点を指定し、その 3 点を通る平面を定義するクラスである。直方体の各頂点の座標より Plane クラスを 6 面に生成し、またその時自動で生成される法線ベクトルにより必要な 2 つの情報を得る。

2.3 本研究の提案手法

先行研究の課題点は以下の三つである。

1. 任意の形状のオブジェクトを操作可能とする。
2. 任意の形状のオブジェクト同士の衝突判定処理を可能とする。

これらを実現するため本研究で実装を計画している内容を以下に示す。

1. MeshCollider を持つオブジェクトの操作の実装。
2. MeshCollider を持つオブジェクトとの衝突判定処理の改善。
3. 拡張機能のウィンドウの UI の改善。
4. Touch の操作によるオブジェクトの編集機能の実装。

3 進捗状況と今後の計画

3.1 進捗状況

まず、進捗状況をまとめた表を表 1 に示す。完了した課題は 100 として表示し、取り組み途中の課題はその進捗を表す。

表 1 進捗状況

	進捗 (%)
1. Mesh オブジェクトの操作	100
2. Mesh オブジェクトの衝突判定の改善	70
3. 拡張機能のウィンドウ UI の改善	100
4. Touch の操作のオブジェクト編集機能	20

3.1.1 Mesh オブジェクトの操作の実装

Mesh オブジェクトの操作に関しては、Mesh オブジェクトの操作時の衝突判定は対応する *Overlap.Sphere* 等の関数がないため、新たな方法を実装した。 *Physics.ComputePenetration* 関数で操作しているオブジェクトのコライダーと接触され

るオブジェクトのコライダーが「重なっているかどうか」を判定し接触しているかの判定を行う方式に変更した。これによって、Mesh オブジェクト以外のオブジェクト（球体や直方体）でも対応した *Overlap.Sphere* 等の関数を用いずとも同じ方式で接触判定が可能になった。

3.1.2 Mesh オブジェクトの衝突判定改善

卒業研究では Mesh オブジェクトとの衝突時の触覚処理には 3.2.2 で述べた Plane クラスを用いていた。しかしこの手法では複雑なオブジェクトになるほど Plane クラスで定義する面の数が増え触覚提示時のベクトルのブレが大きく使用感を悪くさせていた。そのため本研究では SDF（符号付き距離場）[4]を用いる。SDF とは符号付き距離場の略称であり、空間中の任意の点が「物体の表面からどれくらい離れているか」を符号付きの距離で表す関数・データ構造である。この手法によって得られる距離から勾配を求めることで法線ベクトルも求められるため、触覚提示が可能になる。進捗として、実装済みであるが複雑なオブジェクトだとブレが大きいためもうひとつ改良が必要である。

3.1.3 拡張機能のウィンドウの UI の改善

UI の改良について改良前と改良後の二つの UI を図 5 に示す。先行研究の UI ではスタート地点を指定し、その地点からオブジェクトを操作していた。そのため一度設置したオブジェクトをもう一度移動しようとするときスタート地点に戻ってしまうという点が操作性を悪くしていた。この問題を解消するため、オブジェクトの位置から操作を可能に改良した。また、ユーザーが直感的にオブジェクトの操作ができるように、一つのボタンでオブジェクトの操作を開始できるように改良した。

3.1.4 Touch 操作のオブジェクト編集機能の実装

ゲームステージの制作中にユーザーはオブジェクトの移動や回転だけでなく、オブジェクトのサイズの変更や形状の編集ができる機能を実装する。先行研究のシステムではオブジェクトの移動と回転しかできないため、サイズの変更や形状の編集も Touch による操作で可能にすることでより利便性のある拡張機能になると考える。エディタ上のボタンを押すことでオブジェクトの操作モードと編集モードに切り替えられるシステムを制作することを考えている。今回はオブジェクトの拡大縮小機能を追加し

た。今後、オブジェクトの自由編集機能を実装を考えている。



図 5 拡張機能のウィンドウ UI の改良

3.2 今後の計画

今後の計画をまとめた表を表 1 に示す。

表 2 今後の計画表

		7 月		
1. Mesh オブジェクトの操作		完了		
2. Mesh オブジェクトの衝突判定改善				
3. 拡張機能のウィンドウ UI の改善		完了		
4. Touch 操作のオブジェクト編集機能				
5. 評価実験、修士論文執筆				
	8 月,9 月	10 月,11 月	12 月,1 月	
1				
2				
3				
4				
5				

参考文献

[1] ” 産業分野におけるゲームエンジン活用”, シリコンスタジオ ,
<https://tech.siliconstudio.co.jp/solution/game-engine/> (参照 2024-05-22)

[2] 3D Systems Touch ハプティクスデバイス, 3D Systems Touch,
<https://ja.3dsystems.com/haptics-devices/touch> (参照 2024-05-22)

[3] Unity Technologies, Unity,
<https://unity.com/ja> (参照 2024-08-31)

[4] ” 物体認識手法 符号付距離関数 (SDF) ”, 構造計画研究所,
https://www.sbd.jp/column/powder_vol17_sdf.html (参照 2026-04-14)